

QTagger+: Quantum Machine Learning for Ransomware Tagging and Classification with Generative Data Augmentation

Mentors: Dr. Simranjit Singh, Dr. Mohit Sajwan

Project Overview

This project is a continuation and the next edition of QTagger, originally developed under QIntern 2025. The previous edition established a foundational quantum pipeline comprising Quantum Support Vector Machines (QSVM), Variational Quantum Circuits (VQC), and Quantum Kernel methods for binary ransomware classification. This edition introduces two targeted enhancements: generative data augmentation through deep generative models and an extension from binary to multi-family malware classification across five distinct classes. Interns joining this edition will inherit and extend a working codebase, enabling deeper research contributions within the same 6-week window.

Ransomware remains one of the most pervasive and rapidly evolving cybersecurity threats today, with traditional machine learning models increasingly challenged by growing data size, severe class imbalance, and multi-family complexity. This project proposes the use of Quantum Machine Learning (QML), with a focus on both purely quantum and hybrid classical-quantum models, to classify malware across multiple families using large-scale malware datasets. The proposed project utilises frameworks like Qiskit, PennyLane, and TensorFlow Quantum, and explores techniques such as QSVM, VQC, and Quantum Kernel estimation. Additionally, generative model-based synthetic data generation is incorporated to address class imbalance across five malware families: ransomware, trojan, worm, spyware, and benign.

Interns will work on data processing, generative augmentation, feature extraction, model building, performance evaluation, and final reporting. Furthermore, interns will develop a strong foundation in scientific research practices, quantum computing, and generative AI throughout the engagement.

Objectives

- Develop and benchmark quantum and hybrid models for multi-family malware classification across five classes: ransomware, trojan, worm, spyware, and benign.
- Construct a curated ransomware API call behavioural dataset by executing ransomware samples in a controlled sandbox environment, following the dataset construction guidelines established by MLRan (FC Onwuegbuche), to serve as an additional quantum-ready data source.
- Introduce generative model-based synthetic data augmentation to address class imbalance in malware datasets, as an alternative to SMOTE-based oversampling.

- Compare the performance of synthetically augmented versus SMOTE-balanced training pipelines on quantum models.
- Evaluate the feasibility of quantum techniques on large, high-dimensional, multi-class datasets.
- Compare performance with classical machine learning models.
- Train interns in scientific writing, quantum computing, generative AI, and applied cybersecurity AI.

6-Week Plan

Week 1: Orientation and Literature Foundation

- Introduction to ransomware datasets, malware family taxonomies, and multi-class classification challenges.
- Onboarding with quantum programming tools (Qiskit, PennyLane, Cirq) and an introduction to deep generative models for data synthesis.
- Guided review of foundational research papers on QML, hybrid architectures, and generative data augmentation for imbalanced datasets.
- Intern deliverables: annotated summaries of two key papers.

Week 2: Dataset Preparation, Preprocessing, and Generative Augmentation

- Load and clean datasets such as EMBER, CIC-MalMem2022, or custom malware datasets with family labels.
- Construct a ransomware API call behavioural dataset by executing ransomware samples in a sandboxed environment (e.g., Cuckoo Sandbox), capturing system API call sequences, registry operations, and file system interactions. The dataset construction follows the GUIDE-MLRan methodology (Caviglione et al., 2025), which provides a standardised framework for curating high-quality behavioural ransomware datasets.
- Feature extraction (e.g., API call traces, opcode sequences, binary entropy).
- Apply dimensionality reduction techniques (e.g., PCA/UMAP) for QML feasibility.
- Implement a suitable deep generative model to synthesise samples for minority malware family classes.
- Deliverables: Preprocessed dataset notebook, curated API call dataset, and a generative augmentation report comparing synthetic versus SMOTE-balanced class distributions.

Week 3: Classical Baseline and Quantum Model Design

- Train and evaluate classical models (Random Forest, XGBoost, CNN) on 5-class malware classification.

- Design hybrid models: classical preprocessing combined with a VQC or QSVM layer for multi-class output.
- Start coding basic quantum pipelines using simulators.
- Deliverables: Baseline multi-class performance summary (accuracy, F1 per class); initial QML pipeline sketch.

Week 4: Model Implementation and Evaluation

- Implement and test full QML and hybrid models for 5-class malware classification.
- Measure accuracy, F1, and ROC-AUC per class; compare results against classical models.
- Evaluate model performance across three data conditions: synthetically augmented, SMOTE-balanced, and original imbalanced datasets.
- Identify scalability limits and propose enhancements.
- Deliverables: QML model codebase and preliminary results notebook.

Week 5: Optimization and Reporting

- Tune hyperparameters and perform validation experiments.
- Draft initial version of the research paper and implementation documentation.
- Visualise performance comparisons across augmentation strategies and classical benchmarks.
- Deliverables: Draft paper and optimized models.

Week 6: Final Review and Presentation

- Finalize paper, code, and documentation.
- Internal project presentation and peer review.
- Discuss future directions, possible conference submissions, and publication planning.

Mentorship and Guidance Plan

As the research mentors, a structured and interactive learning experience is provided, tailored to both technical depth and academic research readiness:

1. Curated Learning Material

- A dedicated reading list is shared, including foundational papers and tutorials on quantum kernels, quantum SVMs, hybrid neural architectures, and generative models for synthetic data augmentation.
- Additional resources cover scientific writing and implementation documentation practices.

2. Weekly Workshops

- Interactive sessions are conducted each week focusing on:
- Effective scientific communication and paper writing.
- Code documentation and reproducibility standards.
- Interpreting experimental results for publication.

3. Individual and Group Mentorship

- Each intern is assigned a 15-20 minute weekly check-in to address individual technical hurdles, receive focused feedback, and ensure alignment with project goals.
- These stand-ups additionally serve as a forum for cross-learning and collaboration among interns.

4. Timely Feedback and Progress Tracking

- Written drafts and code contributions are reviewed and commented upon within 24 hours to keep the research pace dynamic and responsive.

5. Collaboration Tools and Reporting

- Interns are guided on using GitHub for version control, documentation practices, and collaborative development.
- Weekly deliverables and retrospectives help in tracking and iterating over the project plan.